Luke Matsunaga

Outline of this report:

- Cover page: Page 1
- My report: Pages 2-7
- Sample run using small dataset: Page 8
- Sample run using large dataset: Page 9
- Additional Example using UCI ML Repository: Page 10
- Source code : https://github.com/ActualCookie88/IntroToAIProjects

Feature Selection with Nearest Neighbor

Luke Matsunaga

Introduction

In this project, I used two feature selection algorithms, forward selection and backward elimination, to select a set of features that produce the highest accuracy in classifying a given dataset. The datasets I tested consist of several instances characterized by multiple features, as well as a class label. Feature selection algorithms are necessary in effectively determining related features, especially for datasets with many instances. Both search strategies utilize leave-one-out-cross validation and a nearest neighbor classifier algorithm.

I chose to use C++ and the full code can be accessed through the GitHub link provided on the Cover Page above.

Nearest Neighbor Classification

The nearest neighbor classification is an algorithm that classifies an instance by comparing it to all other instances in the same dataset.

The algorithm computes the distance between the instance being classified and all other instances using the Euclidean Distance Formula, shown in *Figure 1* below. This formula takes a particular feature subset and, for each instance, searches for the closest instance or "nearest neighbor." The instance being classified then adopts the class label of the closest instance determined by the algorithm.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

Figure 1: Euclidean Distance Formula

Leave-One-Out Cross Validation

Leave-one-out-cross validation (LOOCV) is used to estimate the accuracy of a feature subset in terms of classification, assisted by the nearest neighbor classification.

This algorithm iterates through every instance in a given dataset, and temporarily removes one instance as a test instance, using the remaining instances to find the nearest neighbor. If the class label of the test instance and the nearest neighbor match, the classification is correct. This is repeated for all instances, and the number of correct classifications is accumulated to compute the classification accuracy of the feature subset.

Forward Selection

The Forward Selection algorithm starts with an empty feature set and adds a feature at each iteration/level of the search tree. For each level, the algorithm considers and tests the accuracy of possible sets for every added feature if it does not already exist in the set.

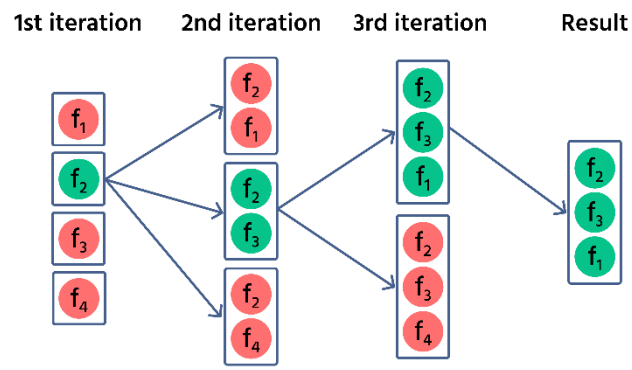

Figure 2: Forward Selection Algorithm using five features

Most importantly, it determines the classification accuracy of temporarily adding each feature to the current set using LOOCV. The feature that produces the highest accuracy is selected and permanently added to the current set. This selection method is repeated until no iterations remain, or when all features have been tested.

Figure 2 illustrates the selection process; it starts with the most accurate feature f_2 in the 1st iteration, then considers three possible sets by adding one of the three other features. In the 2nd iteration, the algorithm determines that set $\{f_2, f_3\}$ produces the highest accuracy among the other sets and repeats the process in the 3rd iteration. This repeats until all iterations have been completed, which gives the algorithm all information necessary to determine the subset with the highest accuracy.

Backward Elimination

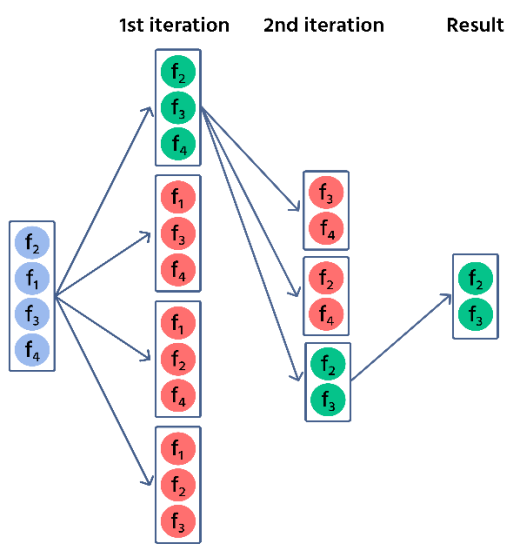

Figure 3: Backward Elimination Algorithm using four features

In contrast to Forward Selection, Backward Elimination starts with the full feature set, and rather than adding a feature at each level, this algorithm removes a feature instead. This algorithm also tests possible subsets for classification accuracy when removing a feature at every level.

Backward Elimination temporarily removes a feature from the current set and evaluates its classification accuracy using LOOCV. The removal that produces the highest accuracy is selected and the associated feature is permanently removed from the current set. This elimination method is repeated until one feature remains.

Figure 3 demonstrates the elimination process; it starts with the full feature set {f_2, f_1, f_3, f_4} and considers four possible sets by removing one of the four features in the 1st iteration. The algorithm determines that removing feature f_1 produces the highest accuracy, so it permanently removes it and moves to the next iteration. This repeats until all iterations have been completed, as a subset with the highest accuracy is not necessarily the last one tested.

Performance of Algorithms on Small Datasets

In *Figure 4* below, I ran the Forward Selection algorithm on CS170_Small_DataSet__7. This file is a dataset that contains 16 features and 500 instances, with each instance having a class label.

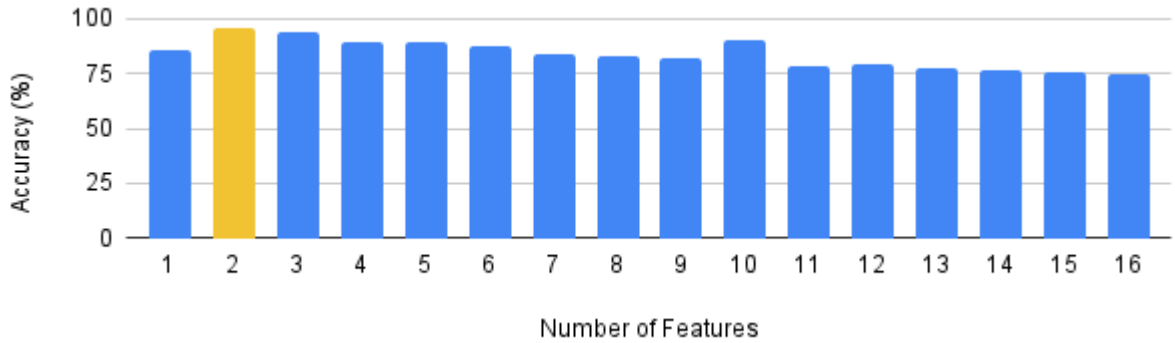

Figure 4: Shows the relationship between classification accuracy and the number of features selected when using the Forward Selection algorithm on a small dataset.

Starting with feature {6} as the first feature, the accuracy is 86%, but adding feature {15} to the current set produced a new accuracy of 95.8%. Next, after adding feature {2} to the current set of {6, 15}, the accuracy slightly reduced to 94%. The accuracy continues to decrease for every added feature from this point on until the last set, which suggests that the remaining features are redundant. The highest reported accuracy from this dataset using Forward Selection was 95.8% with subset {6, 15} as indicated by the gold bar in *Figure 3*.

Additionally, in *Figure 5* below, I ran the Backward Elimination algorithm on the same CS170_Small_DataSet__7 file used above.

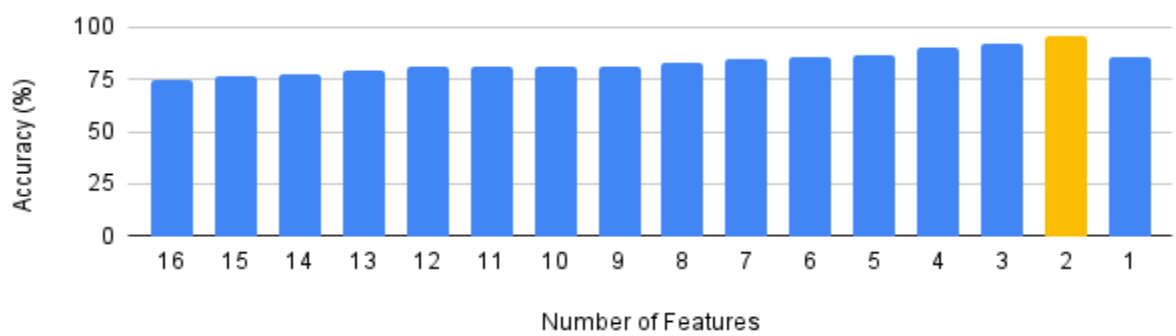

Figure 5: Shows the relationship between classification accuracy and the number of features selected when using the Backward Elimination algorithm on a small dataset.

To confirm the accuracy of the Backward Elimination algorithm, we first determine if the accuracy of its first tested set matches that of the last tested set in the Forward Selection algorithm, as both contain all sixteen features and therefore produce the same classification accuracy. Both algorithms report 74.8%, which is a success. Similarly, the accuracy of Backward Elimination's last tested set must match Forward Selection's first tested set, as both contain one feature and therefore produce the same classification accuracy. Both algorithms report 85%, so we may proceed.

The starting set's accuracy is 74.8%, as mentioned earlier, and the accuracy continues to slightly increase for every removed feature from this point on until the last subset of feature {6}. The highest reported accuracy from this dataset using Backward Elimination was also 95.8% with subset {6, 15}, as indicated by the gold bar in *Figure 4*.

Conclusion for Small Dataset

According to the experiments on the small dataset, both Forward Selection and Backward Elimination successfully determined the optimal subset of features. The most accurate subset was {6, 15}, with a classification accuracy of 95.8%. Both algorithms produced consistently similar accuracies for every subset of features, which supports the conclusion that subset {6, 15} is the best candidate.

Performance of Algorithms on Large Datasets

In Figure 6 below, I ran the Forward Selection algorithm on CS170_Large_DataSet__85. This file is a dataset that contains 64 features and 3000 instances, with each instance having a class label.

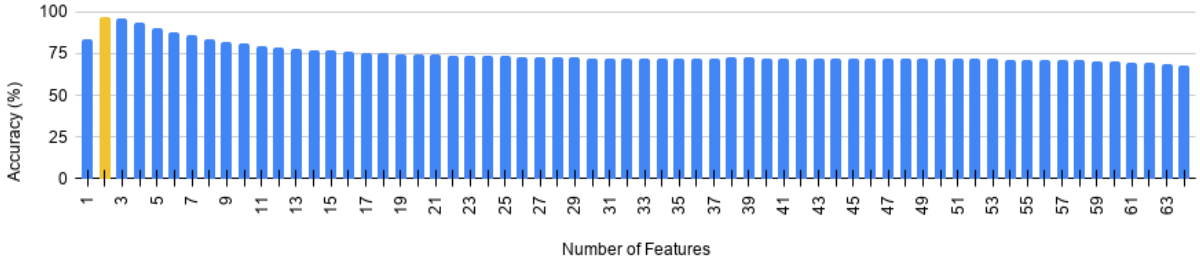

Figure 6: Shows the relationship between classification accuracy and the number of features selected when using the Forward Selection algorithm on a small dataset.

Starting with feature {50} as the first feature, the accuracy is 83.8%, but adding feature {63} to the current set produced a new accuracy of 96.8%. Next, after adding feature {38} to the current set of {50, 63}, the accuracy only slightly reduced to 96%. As shown in *Figure 6*, the accuracy begins the decline for every new feature added to the current set, eventually plateauing around 75% after 15 features. Again, this suggests that the remaining features are redundant, though feature {38} may be a viable candidate. The

highest reported accuracy from this dataset using Forward Selection was 96.8% with subset {50, 63} as indicated by the gold bar in *Figure 6*.

Additionally, in *Figure 7* below, I ran the Backward Elimination algorithm on the same CS170_Large_DataSet__85 file used above.

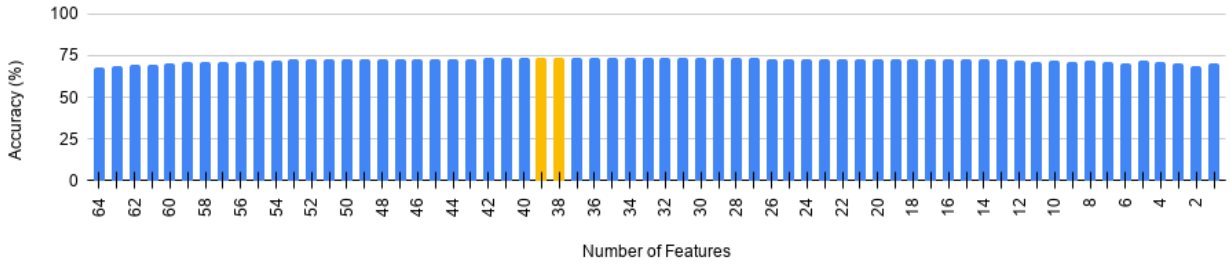

Figure 7: Shows the relationship between classification accuracy and the number of features selected when using the Backward Elimination algorithm on a large dataset

Surprisingly, Backward Elimination performed significantly worse compared to Forward Selection when used on the same dataset, in contrast to the small dataset experiment used earlier. Although the highest accuracy was found in feature subsets of 38 and 39, shown by the two gold bars in *Figure 7*, removing a feature at every level in this dataset showed very little difference. Each subset never dropped below 68% or above 74%.

In addition, Backward Elimination and Forward Selection reported the same accuracy for the 64-feature set (68.1%), but a different accuracy and feature for the 1-feature subset. Forward Selection reported an accuracy of 83.8% using feature {50}, while Backward Elimination reported 70.5% using feature {22}. This helps provide evidence to the discrepancy illustrated in this experiment; since Backward Elimination starts with larger feature subsets, it requires much more LOOCV computations compared to Forward Selection. Furthermore, Backward Elimination is often burdened with many redundant or noisy features that affect the search process in the early stages, while Forward Selection possesses more freedom in introducing the best feature at each step.

Conclusion for Large Dataset

Forward selection significantly outperformed backward elimination in this dataset, as it determined an optimal 2-feature subset with an accuracy of 96.8%. Backward elimination, however, converged to larger subsets of 38-39 features with a lower accuracy of 73.97%, demonstrating the difficulty of removing large numbers of irrelevant features using a greedy search strategy. The results from these experiments demonstrate the danger of datasets containing many irrelevant features and the importance of choosing the most effective search strategy. Backward Elimination likely excels in evaluating datasets that contain more informative features, which would have been the case in a different situation.

Computational Effort and Runtime Results

Both algorithms were implemented in C++ using a laptop with an Intel Core i7-13650HX processor. *Figure 8* below shows the runtime of each algorithm for both datasets. As described earlier, the significantly larger runtime for Backward Elimination, especially for larger datasets, can be attributed to the higher cost of evaluating feature subsets early in the search process.

Algorithm vs. Dataset Size	Small Dataset	Large Dataset
Forward Selection	~1.1 seconds	~30 minutes (0.5 hour)
Backward Elimination	~1.6 seconds	~150 minutes (2.5 hours)

Figure 8: Runtime Comparison of Forward Selection and Backward Elimination on Small and Large Datasets

Sample run using Backward Elimination on small dataset (CS170_Small_DataSet__7)

Feature Selection Algorithm

Choose dataset:

(1) Small Dataset

(2) Large Dataset

(3) Custom Dataset

1

Precomputing feature distances...

This dataset has 16 features with 500 instances.

Choose algorithm:

(1) Forward Selection

(2) Backward Elimination

2

On level 16 of the search tree

Using feature(s) {1,2,3,4,5,6,7,8,9,10,11,12,13,14,15,16} accuracy is 74.8%

(…13 other levels…)

On level 2 of the search tree

Feature set {6,15} was best, accuracy is 95.8%

On level 1 of the search tree

Feature set {6} was best, accuracy is 86%

Finished search. Best feature subset is {6,15} with accuracy 95.8%

Program runtime: 1.56 seconds

Sample run using Forward Selection on large dataset (CS170_Large_DataSet__85)

Feature Selection Algorithm

Choose dataset:

(1) Small Dataset

(2) Large Dataset

(3) Custom Dataset

2

Precomputing feature distances...

This dataset has 64 features with 3000 instances.

Choose algorithm:

(1) Forward Selection

(2) Backward Elimination

1

On level 1 of the search tree

Feature set {50} was best, accuracy is 83.8333%

On level 2 of the search tree

Feature set {50,63} was best, accuracy is 96.8%

(…61 other levels…)

On level 64 of the search tree

Feature set {50,63,38,14,11,53,15,48,57,26,41,17,46,52,13,22,44,29,5,25,20,64,23,54,49,30,51,18,60,16,55,35,6,4,7,39,43,12,40,42,45,36,32,33,9,1,59,58,3,61,8,62,34,10,28,24,37,19,31,21,2,27,56,47} was best, accuracy is 68.1%

Finished search. Best feature subset is {50,63} with accuracy 96.8%

Program runtime: 4515.92 seconds

Additional Example using UCI ML Repository

I chose to use the UCI Wine Dataset[1] containing 178 samples of wines derived from three different classes, each characterized by 13 chemical analysis features. Realistically, a researcher could measure the chemical composition of a certain wine and use the features to classify the wine's origin.

The default rate was calculated to be 39.9%. Using Forward Search on this dataset, the first feature added was feature {7}, Flavonoids, which produced an accuracy of 69.7%. This is accurate, as flavonoid amount in wine can determine its color, tannins and mouthfeel, so this feature can be used to distinguish between three different cultivars. The next feature added was feature {10}, Color Intensity, which produced a higher accuracy of 92.7%. This is also accurate, as color intensity in wine can be a visual indicator of its grape variety, which can help distinguish between the given classes.

The third feature added was feature {2}, Malic Acid, which increased accuracy to 94.4% followed by feature {13}, Proline, which increased accuracy to 95.5%. More features, such as Hue {11}, Alcohol {1}, and Magnesium {5} continued to improve the classification accuracy up to around 98%. These features make sense, as they either contribute to the taste profile of the wine or provide insight into its grape variety.

The accuracy began to plateau around 98%, eventually peaking at 98.9% for a set containing 8 features. From this point on, adding more features either did not increase the accuracy or slightly reduced it, which suggests that the remaining features are likely redundant, or simply less useful in distinguishing wine class. This pattern supports the idea that adding too many features to a subset may introduce unnecessary noise, which ultimately decreases classification accuracy.

Accuracy (%)	# Features	Feature Set
69.6629	1	{7}
92.6966	2	{7,10}
94.382	3	{7,10,2}
95.5056	4	{7,10,2,13}
96.6292	5	{7,10,2,13,11}
97.7528	6	{7,10,2,13,11,1}
98.3146	7	{7,10,2,13,11,1,5}
98.8764	8	{7,10,2,13,11,1,5,6
98.8764	9	{7,10,2,13,11,1,5,6,9}
98.3146	10	{7,10,2,13,11,1,5,6,9,4}
98.3146	11	{7,10,2,13,11,1,5,6,9,4,12
97.7528	12	{7,10,2,13,11,1,5,6,9,4,12,3}
96.0674	13	{7,10,2,13,11,1,5,6,9,4,12,3,8}

Figure 1: Forward Selection results on the normalized wine dataset showing classification accuracy as features are added. Also shows associated feature sets at each step.

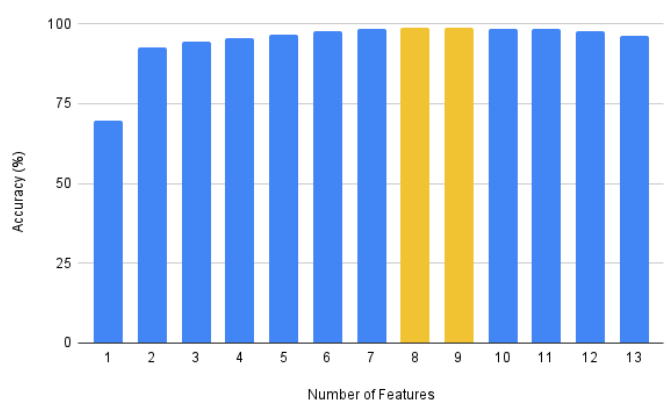

Figure 2: Shows the relationship between classification accuracy and the number of features selected when using the Forward Selection algorithm on the wine dataset.

[1] https://archive.ics.uci.edu/dataset/109/wine